Lab: Team Board

Lifecycle, BuildContext, keys, Row, Column, Expanded, Container and Stack

Duration	90 minutes
Format	Individual work. Every student has personalised numbers.
Slides	Widgets, Layout & Navigation lecture, slides 4 to 19 (Ticking Clock through Stack)
Tools	Flutter SDK (`flutter run`, `flutter analyze`), VS Code, Git, your existing course repository on GitHub
Submission	`git push` to your course repo after every step (10 pushes), plus a final push with answers
Total marks	100

Learning Objectives

By the end of this lab you will be able to:

- Start a `Timer` in `initState`, stop it in `dispose`, and count how often `build` runs
- Tell apart a Widget (new on every rebuild) and an Element (kept), and read a `BuildContext`
- Build a widget by composition and explain why it needs no subclass
- Use `ValueKey` so that State follows its widget when widgets are re-ordered
- Choose `mainAxisAlignment` and `crossAxisAlignment` for a `Row` and a `Column`
- Read an overflow error and fix it with `Expanded`; tell `Expanded` from `Flexible`
- Use `Container` (margin, padding, decoration) and `Stack` with `Positioned`

Your Personal Version of This Lab

Every student gets **different numbers**, so every student's app looks and behaves differently. The starter code reads the **last two digits of your roll number** and turns them into a seed:

- seed is the two-digit number (00 to 99). For roll number 21F-0347 the seed is 47.
- t is the tens digit (4) and u is the units digit (7).

Your clock values, your tile names, your **Row and Column alignment**, your flex numbers, the size and colour of your box and the position of your stamps are all calculated from t and u. The starter shows them on screen in a text block called **YOUR SETTINGS**. An app that is correct for another roll number will **not** match yours, and that is intended.

Rules: do not edit the seeded settings in the starter file; never use `t` or `u` as names of your own variables; keep every `Key(...)` and every text format exactly as shown. For the two axis settings in Step 6 you must **type the value yourself** (for example `MainAxisAlignment.spaceEvenly`), reading the name from your YOUR SETTINGS block. Your instructor checks your app with an automatic test generated for your own roll number, so a text that is one character different counts as wrong.

Time Plan and Checkpoints

Each step ends with a **checkpoint**: you commit and push. The "finish by" column is the minute (from the start of the lab) by which your push should be on GitHub. GitHub records when each push arrived.

Checkpoint	Topic	Time	Finish by
Setup	Create the project, first push	8 min	min 8
Step 1	`Clock`: `Timer` in `initState`	7 min	min 15
Step 2	`dispose`, counting `build`, lifecycle log	7 min	min 22
Step 3	`BuildContext`: Widget vs Element	7 min	min 29
Step 4	Composition and the `Tile`	7 min	min 36
Step 5	Keys: swapping tiles	6 min	min 42
Step 6	`Row` and `Column`: the two axes	8 min	min 50
Step 7	Overflow and `Expanded`	5 min	min 55
Step 8	`Expanded` vs `Flexible`	7 min	min 62
Step 9	`Container`: margin, padding, decoration	6 min	min 68
Step 10	`Stack` and the whole screen	7 min	min 75
Wrap-up	Analyse, answers, screenshot, final push	15 min	min 90
Total		**90 min**	

How to submit: your GitHub repository

You already have a repository for this course. Do **not** create a new one. Put everything for this lab in a Flutter project named `layout_lab` at the top level of that repository:

```
your-course-repo/
  layout_lab/
    lib/main.dart          <- your program (one file)
    ANSWERS.md             <- numeric and written answers (at the end)
    screenshot.png         <- a screenshot of your running app (at the end)
```

At the end of **every step**, run these commands from the top of your repository (replace N):

```
git add layout_lab
git commit -m "Layout lab: step N"
git push
```

Integrity rules: never force-push, rebase or amend a commit that you already pushed. Your commit history is your proof that you did the work in order and in time. Commit messages must follow the format above.

Setup (8 min)

- Open a terminal in your course repository and run `git pull` so you start up to date.
- Run `flutter create layout_lab`. Open the new folder in VS Code.

- Delete the file `layout_lab/test/widget_test.dart` (it tests the counter app that you are about to replace).
- Replace the whole content of `layout_lab/lib/main.dart` with the starter code below (your instructor also shares it as `layout_lab_starter.dart`).
- Type **your name** and **your roll number**, which must end in two digits.
- Run the app (`flutter run -d chrome`, or an emulator or your phone). You should see the title `Team Board: <your course>` and a text block called **YOUR SETTINGS** with your own numbers. Keep it open: you will read your values from it in every step.
- Commit with the message `Layout lab: setup` and push.

```
// main.dart - Team Board lab
import 'dart:async';

import 'package:flutter/material.dart';

const String studentName = 'YOUR NAME HERE';
const String rollNo = 'YOUR-ROLL-NO-HERE'; // e.g. '21F-0347'

// ===== Seeded settings (generated from YOUR roll number). Do not edit. =====
final int seed = int.parse(rollNo.substring(rollNo.length - 2));
final int t = seed ~/ 10; // tens digit
final int u = seed % 10; // units digit

const List<String> courses = [
  'Mobile Dev', 'Data Structures', 'Operating Systems', 'Databases',
  'Networks', 'Software Eng', 'AI', 'Web Engineering', 'Graphics', 'Security',
];
const List<String> names = ['Ali', 'Sara', 'Omar', 'Hina', 'Zain', 'Maya'];
const List<String> mains = [
  'start', 'center', 'end', 'spaceBetween', 'spaceAround', 'spaceEvenly',
];
const List<String> crosses = ['start', 'center', 'end'];
const List<String> hues = ['teal', 'indigo', 'orange', 'pink', 'green'];
const Map<String, MaterialColor> swatches = {
  'teal': Colors.teal,
  'indigo': Colors.indigo,
  'orange': Colors.orange,
  'pink': Colors.pink,
  'green': Colors.green,
};

final String course = courses[u];

// Steps 1-2: the clock
final int startSec = 5 * (1 + t % 5); // where the clock starts
final int tick = 1 + u % 3; // how much it adds every second
final int ticks = 4 + u % 4; // how many ticks until it stops
final int stopAt = startSec + ticks * tick; // the clock stops here

// Steps 4-5: the tiles
final String tileA = names[u % 6];
final String tileB = names[(u + 2) % 6];
final String tileC = names[(u + 4) % 6];
final int tapsToMake = 2 + t % 3; // used by the experiment in Step 5

// Step 6: axes (the NAMES are printed on the settings card; you type the enum)
final String mainName = mains[u % 6];
```

```
final String crossName = crosses[t % 3];

// Step 8: Expanded and Flexible
final double fixedW = 70.0 + 10 * u;
final int flexA = 1 + t % 3;
final int flexB = 1 + u % 2;

// Step 9: Container
final double boxMargin = 4.0 + 2 * (t % 5);
final double boxPad = 8.0 + 2 * (u % 5);
final double boxRadius = 4.0 * (1 + (t + u) % 4);
final double boxBorder = 1.0 + t % 3;
final String hue = hues[(t + u) % 5]; // use swatches[hue]!.shade100

// Step 10: Stack
final double dotTop = 4.0 + 4 * (t % 4);
final double dotRight = 4.0 + 4 * (u % 4);
final double dotBottom = 4.0 + 4 * ((t + u) % 4);
final double dotLeft = 4.0 + 4 * ((t + 2 * u) % 4);
// ============================================================================

// Step 2 writes to this list from initState and dispose.
final List<String> lifecycleLog = <String>[];

void main() => runApp(const LabApp());

// Keep the name LabApp and its const constructor: the checker uses them.
class LabApp extends StatelessWidget {
  const LabApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(title: 'Team Board', home: HomePage());
  }
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Team Board: $course')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SettingsCard(),
            const SizedBox(height: 12),
            // Add each new widget BELOW the previous one, in the order the
            // steps tell you, with a SizedBox(height: 12) between widgets.
          ],
        ),
      ),
    );
  }
}
```

```
// ---- Provided: shows YOUR numbers. Keep it on the page. Do not rename.
class SettingsCard extends StatelessWidget {
  const SettingsCard({super.key});

  @override
  Widget build(BuildContext context) {
    return Text(
      'YOUR SETTINGS (seed $seed)\n'
      'clock: start $startSec, tick $tick, stop at $stopAt\n'
      'tiles: $tileA, $tileB, $tileC (taps for the Step 5 experiment:
$tapsToMake)\n'
      'row main axis: $mainName | column cross axis: $crossName\n'
      'flex: $flexA and $flexB | fixed width: ${fixedW.toStringAsFixed(0)}\n'
      'box: margin $boxMargin, padding $boxPad, radius $boxRadius, '
      'border $boxBorder, colour $hue\n'
      'stamp: top $dotTop, right $dotRight, bottom $dotBottom, left $dotLeft',
      key: const Key('settings'),
      style: const TextStyle(fontFamily: 'monospace', fontSize: 12),
    );
  }
}

// ---- Provided for Step 7. It overflows on purpose. Do not rename anything.
class SummaryRow extends StatelessWidget {
  const SummaryRow({super.key});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text(
          'Part A: read the widget tree of the $course board from top to
bottom.',
          key: const Key('sum-a'),
        ),
        Text(
          'Part B: build each box from small widgets and never extend a built-
in class.',
          key: const Key('sum-b'),
        ),
        Text(
          'Part C: break the layout on purpose, then fix it with Expanded or
Wrap.',
          key: const Key('sum-c'),
        ),
      ],
    );
  }
}
```

If you get a FormatException straight away, you have not replaced the placeholder roll number, or it does not end in two digits. Write all your widgets in this one file, below the starter code. Keep the class names given in each step, because the checker looks for them. Every widget that has a **Key** below must have exactly that key, for example `Text(..., key: const Key('clock'))`. The starter already gives you `LabApp`, `HomePage`, `SettingsCard` and `SummaryRow`; you add the other widgets and place them in the `Column` of `HomePage`, **each one below the previous one**, with a `SizedBox(height: 12)` between them.

Step 1: The Clock: Timer in initState (7 min)

Slides 4 and 5. A StatefulWidget that starts a Timer once, in initState, and calls setState on every tick.

Task 1.1

Write `class Clock extends StatefulWidget` (const constructor with `super.key`, `createState()` returns `_ClockState()`). In `_ClockState` declare two fields: `late Timer _timer;` and `int _seconds = startSec;`.

Task 1.2

Override `initState`. Call `super.initState()` first, then start a periodic timer that adds **your** `tick` to `_seconds` once per second, and stops itself when the clock reaches your `stopAt`:

```
_timer = Timer.periodic(const Duration(seconds: 1), (timer) {
  setState(() => _seconds += tick);
  if (_seconds >= stopAt) timer.cancel();   // stop at your limit
});
```

Task 1.3

For now `build` returns only the text below. Add `const Clock()` to the Column of `HomePage`, below the SettingsCard.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Session: $_seconds s', key: const Key('clock'))
```

Self-check: the number starts at your start value, grows by your tick every second, and freezes at your stop value (all three are in YOUR SETTINGS).

Think (answer in a code comment): Why do we start the Timer in `initState` and not inside `build`? What would happen to the speed of the clock if we started it in `build`?

PUSH. Checkpoint 1: commit message `Layout lab: step 1`, then `git push`.

Step 2: dispose, Counting build, the Lifecycle Log (7 min)

Slide 6. initState runs once, build runs many times, dispose runs once. Now measure it.

Task 2.1

Add `dispose()` to `_ClockState`: first `_timer.cancel();`, then `lifecycleLog.add('dispose');`, and last `super.dispose();`. In `initState`, the line right after `super.initState()` is `lifecycleLog.add('initState');`. (`lifecycleLog` is a global list in the starter.)

Task 2.2

Add a field `int _builds = 0;` to the State. The **first line** of `build` is `_builds++;`. Change `build` so it returns a `Column` (`crossAxisAlignment: CrossAxisAlignment.start`) whose children are the clock Text from Step 1 and a second Text:

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Builds: $_builds', key: const Key('builds'))
```

Self-check: Builds shows 1 at the start, goes up by exactly one every second, and stops growing when the clock stops.

Think (answer in a code comment): Why does Builds go up by one per second and not by two? What error would you see in the console if the widget left the screen and `_timer.cancel()` was missing from `dispose`?

PUSH. Checkpoint 2: commit message `Layout lab: step 2`, then `git push`.

Step 3: BuildContext: Widget vs Element (7 min)

Slides 7 and 8. A BuildContext is your address in the tree. The Widget is thrown away on every rebuild; the Element stays.

Task 3.1

Write `class Probe extends StatelessWidget` with a field `final int seconds;` and the constructor `const Probe({super.key, required this.seconds});`.

Task 3.2

`build` returns a `Column` (`crossAxisAlignment: CrossAxisAlignment.start`) with the five Texts below, in this order. Three hints:

- `identityHashCode(object)` is a number that is the same for the same object and different for a different object.
- `this` is the widget; `context` is the Element (it is the widget's address in the tree).
- `context.widget` is the widget this context belongs to, and `context.findAncestorWidgetOfExactType<Scaffold>()` walks **up** the tree and returns the nearest Scaffold, or `null` if there is none.

Task 3.3

In `_ClockState.build` add `Probe(seconds: _seconds)` as the **third** child of the Column. Do **not** write `const` in front of it: its number changes.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Seen at: $seconds s',                    key: const Key('probe-seen'))
Text('Widget #${identityHashCode(this)}',       key: const Key('probe-widget'))
Text('Element #${identityHashCode(context)}',    key: const Key('probe-
element'))
Text('Context belongs to: ${context.widget.runtimeType}', key: const
Key('probe-context'))
Text('Scaffold above me: $above',              key: const Key('probe-above'))
// above = a bool you compute
```

Self-check: every second the Widget number changes but the Element number stays the same; the last two lines say `Probe` and `true`.

Think (answer in a code comment): Which of the two, Widget or Element, is "cheap and thrown away" and which one keeps the State? Where is the third one, the RenderObject, and what is its job?

PUSH. Checkpoint 3: commit message `Layout lab: step 3`, then `git push`.

Step 4: Composition and the Tile (7 min)

Slides 9 to 11. Small widgets wrapped together instead of one new class per look; State lives in the State object.

Task 4.1

Write `class Welcome extends StatelessWidget`. Its `build` returns `Center(child: Text(...))` with the text below. Put `const Welcome()` in `HomePage` **above** the Clock (with a `SizedBox(height: 12)` after it).

Task 4.2

Write `class Tile extends StatefulWidget` exactly as on slide 11: constructor `const Tile(this.label, {super.key});` and a `final String label;`. Its State has `int taps = 0;` and `build` returns an `ElevatedButton` whose `onPressed` does `setState(() => taps++)` and whose child is `Text('${widget.label}: $taps')`.

Task 4.3

In `HomePage`, below the Clock add `Row(children: [Tile(tileA), Tile(tileB), Tile(tileC)])`. Step 5 will move this Row into a new widget.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Welcome, $studentName', key: const Key('welcome'))
Text('${widget.label}: $taps')            // inside the Tile button; no key needed
```

Self-check: tapping a tile raises only its own number: `Sara: 0` becomes `Sara: 1` while the others stay at 0 (your names are in YOUR SETTINGS).

Think (answer in a code comment): You built `Tile` out of an `ElevatedButton` without writing `extends ElevatedButton`. What is this style called, and why does Flutter prefer it?

PUSH. Checkpoint 4: commit message `Layout lab: step 4`, then `git push`.

Step 5: Keys: Swapping Tiles (6 min)

Slides 10 to 12. Flutter matches widgets by type and position unless a key says otherwise.

Task 5.1

Write `class SwapBoard extends StatefulWidget`. Its State keeps the list of tiles, each with a `ValueKey` made from its label, and a method that reverses it:

```
var tiles = [
  Tile(tileA, key: ValueKey(tileA)),
  Tile(tileB, key: ValueKey(tileB)),
  Tile(tileC, key: ValueKey(tileC)),
];
void swap() => setState(() => tiles = tiles.reversed.toList());
```

Task 5.2

`build` returns `Column(children: [Row(children: tiles), <swap button>])`. In `HomePage` **replace** the temporary Row from Step 4 with `const SwapBoard()` (each tile must appear once).

Task 5.3: the experiment

Temporarily **remove the three `key:` arguments**, run the app, tap your **first** tile `tapsToMake` times (see YOUR SETTINGS), press Swap, and look at the **left** tile. Write its number in `ANSWERS.md` as A3. Then **put the keys back** before you commit.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
ElevatedButton(key: const Key('swap'), onPressed: swap, child: const
Text('Swap'))
```

Self-check: with keys, the first tile jumps to the right and takes its count with it; without keys the count stayed on the left while the names swapped.

Think (answer in a code comment): Without keys, why did the number stay at the left even though the label moved? Use the words widget, Element and State in your answer.

PUSH. Checkpoint 5: commit message `Layout lab: step 5`, then `git push`.

Step 6: Row and Column: the Two Axes (8 min)

Slides 13 to 15. A Row's main axis is horizontal; a Column's main axis is vertical; the cross axis is the other one.

Task 6.1

Write `class InfoPanel extends StatelessWidget`. Its `build` returns a `Column` with `crossAxisAlignment: CrossAxisAlignment.start` and three children: a `Row`, a `SizedBox(height: 12)`, and a second `Column`. Add `const InfoPanel()` to `HomePage` below the SwapBoard.

Task 6.2: the Row

Give the Row `key: const Key('info-row')`. Set its `mainAxisAlignment` to **your own value**, read from the line `row main axis:` of YOUR SETTINGS (for example `MainAxisAlignment.spaceEvenly`). **Type it yourself.** Its three Text children are listed below.

Task 6.3: the Column

Give the inner Column `key: const Key('info-col')`. Set its `crossAxisAlignment` to **your own value**, read from `column cross axis:` of YOUR SETTINGS. Its three Text children are listed below.

REQUIRED TEXTS AND KEYS (copy the format exactly)

```
Text('Start: $startSec s',    key: const Key('row-a'))
Text('Tick: $tick s',       key: const Key('row-b'))
Text('Stop: $stopAt s',      key: const Key('row-c'))
Text('Name: $studentName',  key: const Key('col-a'))
Text('Roll no: $rollNo',    key: const Key('col-b'))
Text('Course: $course',     key: const Key('col-c'))
```

Self-check: change your axis value to another one, hot reload, and watch the three Texts move; then set it back to yours.

Think (answer in a code comment): In a Column, which axis is the main axis? Why does `crossAxisAlignment: CrossAxisAlignment.center` in a Column move the children left or right and not up or down?

PUSH. Checkpoint 6: commit message `Layout lab: step 6`, then `git push`.

Step 7: Overflow and Expanded (5 min)

Slide 16. When a Row asks for more width than it has, Flutter paints yellow-black stripes.

Task 7.1

The starter contains `SummaryRow`, which overflows on purpose: a `Row` of three long Texts. Add `const SummaryRow()` to `HomePage` below the InfoPanel and run the app. You see the yellow-black stripes. Open the debug console and read the message: `A RenderFlex overflowed by ... pixels on the right.`

Task 7.2

Fix it: wrap **each of the three Texts** in `Expanded(child: ...)`. Keep the `key` on the Text, not on the Expanded. Do not rename anything.

Self-check: no stripes, the three paragraphs sit side by side in three equal columns, and each one wraps onto several lines.

Think (answer in a code comment): Why does wrapping a Text in `Expanded` make it wrap its words? Use the idea "constraints go down, sizes go up" from slide 13.

PUSH. Checkpoint 7: commit message `Layout lab: step 7`, then `git push`.

Step 8: Expanded vs Flexible (7 min)

Slide 17. The fixed child takes what it asks for; the rest is shared by flex. Expanded must fill its share, Flexible may stay smaller.

Task 8.1

Write `class ShareRow extends StatelessWidget` returning a `Row` with `key: const Key('share-row')` and three children, in this order:

- `SizedBox(key: const Key('fixed'), width: fixedW, child: ElevatedButton(onPressed: () {}, child: const Text('F')))`
- `Expanded(flex: flexA, child: ElevatedButton(key: const Key('exp'), onPressed: () {}, child: const Text('Exp')))`
- `Flexible(flex: flexB, child: ElevatedButton(key: const Key('flex'), onPressed: () {}, child: const Text('Flex')))`

Task 8.2

Add `const ShareRow()` to `HomePage` below the SummaryRow. Then work out, for ANSWERS.md as A4: what **percent** of the leftover width (the width of the Row minus your `fixedW`) does the Expanded button get? Round down to a whole number. The leftover is shared in the ratio `flexA : flexB`.

Self-check: the Exp button stretches to fill its share; the Flex button keeps its natural small size although it also has a share.

Think (answer in a code comment): What would change on the screen if you swapped `Expanded` and `Flexible` between the two buttons? Which one is "Flexible with fit: tight"?

PUSH. Checkpoint 8: commit message `Layout lab: step 8`, then `git push`.

Step 9: Container: Margin, Padding, Decoration (6 min)

Slide 18. Margin is space outside the box, padding is space inside, decoration paints the box.

Task 9.1

Write `class InfoBox extends StatelessWidget` whose `build` returns the `Container` below. Take **your** numbers from the starter variables; the colour comes from the helper map in the starter.

```
Container(
  key: const Key('info-box'),
  margin: EdgeInsets.all(boxMargin),
  padding: EdgeInsets.all(boxPad),
  decoration: BoxDecoration(
    color: swatches[hue]!.shade100,
    borderRadius: BorderRadius.circular(boxRadius),
    border: Border.all(width: boxBorder),
  ),
  child: Text('Course: $course', key: const Key('box-text')),
),
```

Task 9.2

Add `const InfoBox()` to `HomePage` below the ShareRow. **Warning:** a `Container` must never have both `color:` and `decoration:`. Put the colour **inside** the `BoxDecoration`, as above.

Self-check: you can point at the gap outside the border (margin), the gap between border and text (padding), the rounded corners, and the tinted background.

Think (answer in a code comment): Which one is inside the border, margin or padding? When would you use a plain `Padding` or `SizedBox` instead of a `Container`?

PUSH. Checkpoint 9: commit message `Layout lab: step 9`, then `git push`.

Step 10: Stack and the Whole Screen (7 min)

Slide 19. The first child of a Stack is the bottom layer; Positioned pins a child to an edge of the Stack.

Task 10.1

Write `class StampStack extends StatelessWidget` returning a `Stack` with `key: const Key('stack')` and **three** children, in this order (first = bottom layer):

```
Container(key: const Key('layer-base'), width: 300, height: 120, color:
Colors.grey.shade300),
Positioned(top: dotTop, right: dotRight,
    child: const Icon(Icons.star, key: Key('dot-top'))),
Positioned(bottom: dotBottom, left: dotLeft,
    child: const Icon(Icons.circle, key: Key('dot-bottom'))),
```

Task 10.2

Add `const StampStack()` at the bottom of `HomePage`. The page must now show, from top to bottom: SettingsCard, Welcome, Clock (with Probe), SwapBoard, InfoPanel, SummaryRow, ShareRow, InfoBox, StampStack. **Every part appears once.**

Task 10.3

Run `flutter analyze` and add `const` wherever it suggests. Make sure there are no yellow-black stripes anywhere on a small window.

Self-check: the star sits `dotTop` from the top and `dotRight` from the right edge of the grey layer, and the circle sits `dotBottom` from the bottom and `dotLeft` from the left edge of the same layer.

Think (answer in a code comment): What decides the size of the Stack? What happens to the layout if the Icons come first and the grey Container last?

PUSH. Checkpoint 10: commit message `Layout lab: step 10`, then `git push`.

Wrap-up (15 min)

- Run `dart format layout_lab/lib` and `flutter analyze` (inside `layout_lab`). Fix every warning.
- Take a screenshot of your running app and save it as `layout_lab/screenshot.png`. The AppBar title and your YOUR SETTINGS block must be visible.
- Create `layout_lab/ANSWERS.md` with your name and roll number, the **four numbers** below, and short written answers:

```
A1: <the number shown by the Clock when it stops>
A2: <how many times Clock.build runs from the first display until the clock
stops>
A3: <the number on the LEFT tile after Swap, with the keys removed (Step 5
experiment)>
A4: <percent of the leftover Row width that the Expanded button gets, rounded
down (Step 8)>

B1: which of your classes are Stateless and which are Stateful, and why
B2: Widget, Element, RenderObject: which one is rebuilt on every setState and
which ones are kept? What did Probe show you?
B3: explain "constraints go down, sizes go up" using your Step 7 or Step 8
example
```

Write each A answer as a plain whole number (for example `A1: 39`). Your instructor checks A1 to A4 automatically against your roll number. Work them out from your settings and from what you observed in the app. Do not guess: they follow from the numbers in YOUR SETTINGS.

- Final push, commit message `Layout lab: final`:

```
git add layout_lab
git commit -m "Layout lab: final"
git push
```

Common Pitfalls

- `Undefined class 'Timer'`: the import `dart:async` is missing.
- Forgetting `super.initState()` first or `super.dispose()` last.
- Starting the `Timer` inside `build`: a new timer starts on every rebuild and the clock speeds up.
- Writing `const Probe(...)`: a const widget is never rebuilt, so the Widget number would never change.
- `Expanded` placed outside a `Row` or `Column`, for example around a `Text` in a `Stack`: Flutter throws "Incorrect use of ParentDataWidget".

- `Container` with both `color:` and `decoration:`: Flutter throws an assertion. Put the colour in `BoxDecoration`.
- The `key:` put on the `Expanded` instead of the `Text` it wraps: the checker will not find your widget.
- A key written as `Key("x")` with a different spelling, or missing `const`: the checker will not find the widget.
- `Positioned` placed outside a `Stack`: it needs a Stack as its parent.
- Forgetting to put the `ValueKey`s back after the Step 5 experiment.
- Run `flutter pub get` if VS Code shows red imports right after `flutter create`.

Marking Scheme

Section	Marks	What is checked
Steps 1 to 10 (8 marks each)	80	The app passes the automatic test for the step with your own seed (4); the code uses the required feature correctly (3); the step was pushed by its checkpoint (1)
Final project	10	`flutter analyze` is clean, the code is formatted, `screenshot.png` is included, A1 to A4 are correct
Reflection	10	B1 to B3 and the Think comments
Total	**100**	